A support assistant for CloudServe Solutions

Classify, retrieve, draft with citations, and hand every reply to a person

Capstone project report

Sushranth Hebbar

Forward Deployed AI Engineering

November 2026

Repository: SupportTicketsResolver (branch *confidence-calibration*)

Contents

1. Executive summary

2. The problem

3. Discovery findings

4. Requirements

5. Architecture and design

6. Implementation

7. Evaluation

8. Governance and risk

9. The requirements revision

10. Conclusions

Declaration of AI tool use

Appendix A. Prompt register

Appendix B. Full result tables

Appendix C. Requirement traceability

Appendix D. Workbooks, logs and reproduction

Reading note. Every performance figure in this report states what it was measured on. Three data sets are used: 500 development tickets (used for analysis and for choosing prompts and rules), 80 validation tickets (used for the final comparison and not for tuning), and a hidden set of 120 tickets that I was never given. Where something was not measured, the text says so.

1. Executive summary

CloudServe Solutions is a company of about 150 people whose support team of six agents receives more than 500 tickets a week. Customers wait 8 to 12 hours for a first reply against a two-hour agreement, and about 42 percent of tickets are resolved on first contact. The head of support asked for a chatbot that answers the easy tickets so that response times improve without more staff. My discovery work suggested that this was not quite the problem. In the 500 development tickets, 71.4 percent are marked answerable from the existing knowledge base, so the answers largely exist; the difficulty is that agents struggle to find them and that escalated tickets reach specialists without the context they need. A chatbot that answers on its own would also risk the outcome the head of support named as most damaging, which is a confident wrong answer.

I therefore built a support assistant for agents, not a replacement for them. For each ticket it identifies the intent and urgency, retrieves the relevant knowledge-base passages, drafts a reply that cites its sources, and, for the tickets a specialist must handle, prepares an escalation summary. An agent reads every draft before anything could reach a customer, and the system has no code path that sends a message. Every step is written to a decision log, and guardrails block tickets and drafts that contain credential-like or instruction-like text.

The results below come from the 80-ticket validation set, which I ran once in the final configuration. The run processed all 80 tickets with no errors. Intent classification was correct for 91.3 percent of tickets (73 of 80). The expected article was among the retrieved passages for 98.1 percent of answerable tickets (52 of 53). All 14 tickets that policy says must go to a specialist (security incidents, compliance requests, feature requests and unclear requests) were escalated. The decision log reconciled exactly with the tickets processed. Latency was poor: the median was 6.3 seconds and the 95th percentile 55.8 seconds, against a target of 3 seconds, although the run used a rate-limited free model tier and those figures include waiting for the provider. Urgency was predicted correctly for only 43.8 percent of tickets, no better than always answering "medium", so I made urgency advisory: it orders the agent's queue and never changes a decision.

Two further findings shaped the design. First, the model's confidence scores could not be turned into a safe gate for automatic answering: the model reports only three distinct values, and wrong answers occur at the highest of them. The system therefore sends ordinary tickets to review rather than answering them, and the "auto-respond" route means only that a draft is ready for an agent. Second, in the development labels, 67 of 97 groups of identical ticket text carry conflicting urgency labels, which limits what any urgency measurement can show.

The most important caveat is that I did not measure the hallucination rate or citation accuracy. Both need a human to judge whether each claim is supported, and I did not carry out that review. The automatic citation check proves only that a cited document was retrieved. My own informal read of 50 validation drafts found 39 sound, 7 with minor assumptions and 4 that would mislead a reviewer who only skims, and the automatic checks caught none of the four. First contact resolution, reply time and customer satisfaction cannot be measured offline, because the prototype resolves no real tickets, and the hidden 120-ticket set has not been run by me. The figures in this report therefore describe how well the assistant classifies and retrieves, and what it does when things fail; they are not evidence that it improves CloudServe's support outcomes.

2. The problem

2.1 What was asked for

The project brief describes CloudServe Solutions as a company of about 150 people serving just over 200 corporate customers, with a support function that has broken down. Tickets arrive by email, chat, comments on documentation pages and a user forum, more than 500 a week, and are handled by six agents in two tiers. Marcus Adeyemi, the head of support, reports that the first reply takes 8 to 12 hours although the service agreement promises two, that 42 percent of tickets are resolved on first contact against an industry figure he quotes as 65 percent, and that customer satisfaction has fallen to 3.2 out of 5. He asked for a chatbot that answers the easy tickets, so that response times improve without adding staff. He also named what would make the project fail: wrong answers, extra work for agents, poorer service for enterprise customers, and decisions nobody can explain.

I also read the transcripts of four further people who see the problem from different positions. Table 1 summarises what each of the five said and what each could not see.

Table 1. The five stakeholders, what they said, and what they could not see

Person	What they told me	What they could not see
Marcus Adeyemi, head of support	First contact resolution matters more to him than reply time; wants easy tickets answered; fears wrong answers and unexplainable decisions.	The ticket mix. He says any breakdown would be a guess.
Sofia Restrepo, Tier One agent	Handles 40 to 70 tickets a day; familiar tickets take 4 to 5 minutes, unusual ones up to 40; documentation search is so poor that agents keep personal answer files; would welcome a draft with its article attached; believes non-fluent customers fare worst.	Her estimate that 70 percent of tickets are familiar is experience, not a count.
Daniel Okonkwo, Tier Two engineer	About half of the tickets escalated to him could have been resolved at Tier One with the right article; escalations arrive without a summary or notes, so he rereads the thread and repeats questions.	The "about half" is an informed observation, not a measurement.
Ines Varga, technical writer	Maintains 29 articles she considers accurate; believes agents rarely use them because search matches titles and exact terms; wants any system to name its source so errors can be traced.	How often the articles are actually used, and how many tickets they cover.
Ravi Menon, customer	Support is slow but good once reached; waiting matters most when a deployment is blocked; finds the answer himself about half the time; would accept a clearly disclosed machine draft confirmed by a person; worries about a service gap between business and enterprise customers.	Whether his experience is typical.

2.2 What discovery revealed

Each person described a different part of one situation, and the ticket data allowed me to check their claims against each other. The documentation is not the main gap: 71.4 percent of the 500 development tickets are labelled answerable from the existing articles. Yet agents do not use the articles, customers find the answer themselves only about half the time, and 56.2 percent of tickets are escalated, of which 49.1 percent were answerable from documentation. That figure is close to Daniel's "about half". The picture is therefore of a company that has most of the answers but cannot put them in front of the right person at the right moment, and that loses time again when a ticket does need a specialist, because it arrives without context.

This differs from the request in two ways. The request was for automation, and the evidence points to retrieval and preparation: finding the right article, drafting from it, and passing on what has already been tried. And the request implied that the cost of a mistake is small, when the head of support himself names a wrong answer as a way the project fails. Both observations argue for an assistant whose output is reviewed by an agent, not one that replies to customers on its own.

2.3 The gap, stated plainly

CloudServe asked for a system that answers customers. What its own data supports is a system that helps agents answer them: one that finds the article, drafts a cited reply, says when it cannot, and prepares the hand-over for the tickets that need a specialist. The distance between the two is the reason the first reply takes eight to twelve hours and the reason a confident wrong answer would be costly. It also sets the limit of what I could honestly claim. A system that only drafts for review cannot, by itself, change the first contact resolution rate, and I have no pilot data, so I do not claim that it does.

Three further points guided the scope. The project does not rewrite the knowledge base, because retrieval already finds an expected article for nearly every answerable ticket. It does not replace any agent. And it must not widen the service gap that Ravi fears, so every performance figure is examined by customer group, as Section 8 reports.

3. Discovery findings

The discovery evidence has two sources: the five interview transcripts, and the 500 development tickets, each with a label set and a history of how the ticket was handled by people. All figures in this section come from the development tickets unless stated. The full tables are in the Stage 1 workbook, which is included in the submission. I present the findings that changed the design.

3.1 The answers exist, and most escalations did not need to happen

Of the 500 development tickets, 357 (71.4 percent) are labelled answerable from the existing articles. A little over half (56.2 percent) were historically escalated, and 49.1 percent of those escalated tickets were answerable from documentation. This supports Daniel's estimate and weakens the idea that the team lacks answers. It points to the first design decision: the system should retrieve and show the article, and cite it, instead of generating answers from the model's own knowledge. The two historical outcome measures matter for context: first contact resolution is 43.8 percent in this sample, against the 42 percent Marcus quotes, and the satisfaction average is 2.97 out of 5, against the reported 3.2. The difference between those figures and the interview figures may reflect different periods or definitions, and I treat the interview figures as the operational baseline.

3.2 The time goes to a small number of kinds of ticket

I used resolution minutes as a measure of effort. The average ticket takes 421.7 minutes. The tickets that cost the most are the ones a machine should least be trusted with: security incidents average 785.9 minutes and take 9.7 percent of all minutes from 5.2 percent of tickets; feature requests average 723.9 minutes; compliance requests average 679.3 minutes. The cheap tickets are the routine ones: rate limits average 122.2 minutes, single sign-on configuration 191.0 and onboarding 202.1. Figure 1 shows the contrast.

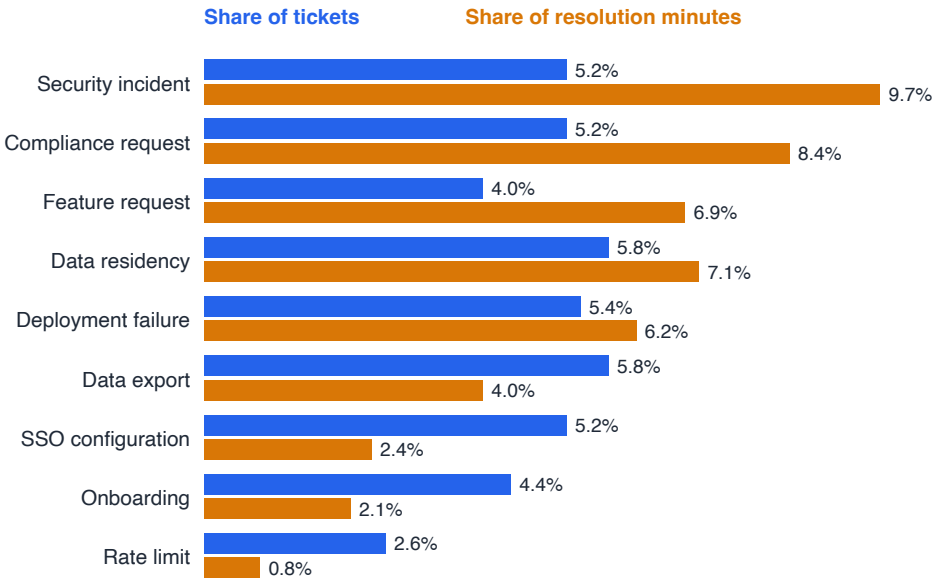


Figure 1. Share of tickets against share of resolution minutes, selected categories (500 development tickets; effort measured as the sum of historical resolution minutes).

Two design consequences follow. Easy, well-documented categories are the right place for a draft to save time, and the hard categories (security, compliance, feature requests, and tickets that are unclear) should go to a specialist with a good summary. Because these four intents also have no

guaranteed answer in the corpus, which Ines noted for feature requests and novel incidents, I made them a standing rule: they always escalate, whatever the model's confidence. Later evidence (Sections 7 and 9) is the reason this rule, not a confidence threshold, became the main safeguard.

3.3 First contact resolution matters more than speed alone

Marcus said first contact resolution matters more to him than reply time, and Ravi said waiting is the main problem. The data relates to both. Average satisfaction is 3.41 out of 5 for tickets resolved at first contact and 2.63 for tickets that were not; it is 3.40 for the fastest quarter of tickets and 2.63 for the slowest quarter. The two are not independent, because a ticket resolved first time is also fast. I concluded that the system should aim at both outcomes, and that satisfaction should be watched so that it does not fall as first contact resolution improves.

3.4 The non-fluent customer hypothesis

Sofia said non-fluent customers take longer and are the least satisfied. The data partly supports her. The 120 non-fluent tickets (24.0 percent) have a mean resolution time of 466.1 minutes against 407.7 for fluent customers. But their satisfaction is 3.04 out of 5, slightly above the 2.97 average, and their first contact resolution of 45.8 percent is slightly above the 43.8 percent overall. So the cost of non-fluency shows up as time, not as unhappiness. I treated it as a fairness risk for the retrieval step, which depends on the customer's phrasing matching the documentation, and tested it explicitly (Section 8).

3.5 Differences by tier and a surprise in the data

Ravi worried about a gap between business and enterprise customers. The data shows enterprise customers with the lowest first contact resolution (37.3 percent against 48.2 for business and 43.1 for standard), the highest escalation rate (62.7 percent) and the longest mean resolution time (483.7 minutes). I did not expect that, and it means that a system which does not widen the gap must be checked for enterprise tickets in particular. The repeat contact rate of 21.6 percent also seemed high to me, and I record it as a measure I could not influence offline.

3.6 What nobody mentioned

No one raised how customer information should be protected when tickets are processed, although every ticket carries a customer name and identifier. That led to the privacy requirement and to the rule that the log, and later the guardrails, must handle credentials and personal data. No one raised differences between channels either; I checked these later and found they largely disappeared once I allowed for the way scenarios cluster (Section 7).

3.7 The problem statement

The statement I wrote at the end of discovery reads: CloudServe's support team is struggling to keep up with more than 500 tickets a week. Customers wait 8 to 12 hours for a first reply, despite the two-hour service promise, and fewer than half of tickets are resolved on first contact. Many answers already exist in the support articles, but agents struggle to find them, and escalations often arrive without clear notes on what has already been tried. Customers wait longer, and agents repeat work.

4. Requirements

I wrote the product requirements document at the end of the first week and revised it after the build; the current version is 2.0 (11 November 2026) and version 1.0 (31 October) is preserved in the repository history. Each requirement has an identifier, a priority and a link to the discovery finding that produced it. Table 2 lists the functional requirements, how each followed from the findings, and what the build delivered. Appendix C gives the full traceability from requirement to code and test.

Table 2. Functional requirements, their origin and their status in the prototype

ID	Requirement (short form)	Priority	Why (finding)	Status
FR-01	Accept a ticket and show its available context for review	Must	Sofia's queue; Daniel rereading threads	Built
FR-02	Find and show relevant documentation	Must	71.4% answerable; Sofia and Ines on search	Built; plan-scope check specified, not built
FR-03	Draft a reply grounded in the displayed documentation	Must	Sofia and Ravi want a source and a draft	Built; subject-versus-body handling specified, not built
FR-04	Agent reviews, edits, rejects or approves before anything is sent	Must	Marcus on wrong answers; Ravi on human confirmation	Built; no send path exists
FR-05	Recognise insufficient evidence; recommend clarification or escalation	Must	Marcus, Sofia, Daniel	Built, including the must-escalate intents
FR-06	Prepare an escalation summary	Should	Daniel on missing context	Built (deterministic summary)
FR-07	Record sources used and the agent's decision	Must	Marcus and Ines on traceability	Built
FR-08	Capture outcomes and agent feedback	Should	First contact resolution as the key measure	Built; no pilot data exists
FR-09	Assign intent and urgency with confidence	Must	Marcus lacks a ticket breakdown; Build Specification A3	Built; urgency advisory only

The non-functional requirements follow from the same evidence. NFR-01 sets a latency target (95th percentile under 3 seconds, from the Evaluation Framework). NFR-02 is availability, whose targets are still to be agreed. NFR-03 sets answer quality: a hallucination rate of 5 percent or lower and citation accuracy of 95 percent or higher, to be established by human review. NFR-04 and NFR-05 cover privacy and auditability, and they came from the thing nobody mentioned in the interviews. NFR-06 sets a fairness test of under 5 percentage points between a customer group and the rest. NFR-07 is cost, with limits still to be agreed. NFR-08 was added in the revision and requires that supplied labels are never rewritten and that prompt and policy decisions are made on development data only.

Some things are deliberately out of scope, and the PRD says why. The system will not send replies to customers on its own, because the head of support's worst case is a confident wrong answer and the customer wants human confirmation. It will not replace agents or rewrite the knowledge base, because most answerable tickets already have an article that retrieval finds. It will not use agents' personal answer files as sources, because Daniel warns they may be wrong. And it will not change tiers, service agreements or staffing levels, which are decisions for CloudServe.

Three of the requirements rest on assumptions that I wrote down in version 1.0 so that the revision would be a considered update. The assumptions were that the documentation is usable as an answer source, that ticket records contain enough context for a useful draft, and that agents will review and edit drafts properly. Section 9 reports which held. Version 1.0 also contained three assumptions I had not written down and which turned out to matter more: that a confidence score can decide which tickets get an automatic answer, that urgency can be predicted from the text and used to route, and that a free model provider can run the whole evaluation unattended. All three were wrong.

The success measures in the PRD are split into those an offline evaluation can report (intent precision and recall, retrieval hit rate, citation proxy, latency and group gaps) and those that need a supervised pilot with real outcomes (first contact resolution, time to first reply, satisfaction). The second group is reported as unavailable, never inferred from the historical baseline. That split is the reason this report cannot say whether the assistant improves CloudServe's first contact resolution.

5. Architecture and design

5.1 The system

Figure 2 shows the path of a ticket. The assistant is a Python service with a command-line evaluation runner and a small web page for agents. A ticket from any of the four channels is first normalised into one format, with the customer fields held separately and the evaluation labels and historical outcomes kept out of anything the model sees. A language model then classifies it, returning an intent from the 22 categories in the data, an urgency, a confidence for each, and for the urgency a short quotation from the ticket as evidence. The router applies a fixed policy to the result. Tickets whose intent is security incident, compliance request, feature request or unclear request go to a specialist; the system retrieves the closest articles and prepares an escalation summary for them. For every other ticket the system retrieves passages, asks the model to draft a reply using only those passages, and runs guardrails over the ticket and over the draft. A ticket that passes becomes a draft waiting for an agent; one that fails, or has too little evidence, becomes a note to a human saying why. Every step writes a row to the decision log.

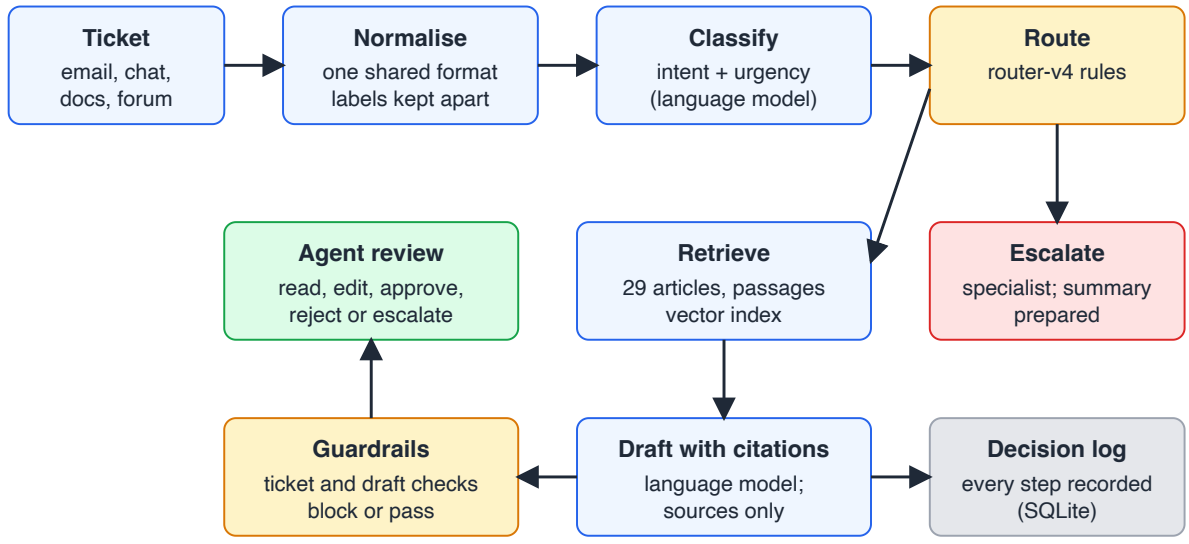


Figure 2. The path of a ticket through the assistant. Everything ends at the agent's review screen; nothing is sent to a customer.

Retrieval uses the 29 supplied knowledge-base articles. Each is split into passages of about 800 characters with a 120-character overlap, embedded with the MiniLM sentence model and stored in a persistent vector index, and the five nearest passages are returned with their document identifiers. The drafting model sees only those passages and the ticket, is told that ticket text is untrusted data, and must place an inline citation such as [DOC-AUTH-004] after each claim. The agent's review page lists the drafts, shows each with its sources, and lets the agent edit, approve, reject or escalate; approving re-runs the citation and safety checks. Observed outcomes and agent feedback have their own store, and a dashboard summarises decisions, guardrail activations and outcomes by customer tier and fluency, with a plain statement that it makes no causal claim. The classifier and the drafter call an OpenAI-compatible chat endpoint, and the evaluation used the free tier of Groq running the Qwen 3.8 27-billion-parameter model. Prompts are versioned PR-02 for drafting and PR-03 for classification.

5.2 Alternatives considered and why I chose as I did

Table 3. Design choices, the alternative, and the reason

Decision	Alternative	Why I chose as I did
Draft for agent review; no send path	An autonomous chatbot that replies directly	The head of support's worst case is a wrong answer and the customer wants human confirmation. The 50 drafts I read included four that would mislead a reviewer who skims; sending them unreviewed would have reached customers.
Answer only from retrieved passages, with inline citations	Letting the model answer from its own knowledge	71.4 percent of tickets are answerable from the articles, and Ines wants any error traceable to either the article or the system.
Four intents always escalate (rule)	Gate every ticket on a calibrated confidence	Confidence did not separate right from wrong answers (Section 7.4). The rule is simple and the labels in the data follow it exactly.
Urgency advisory only	Escalate high-urgency or ambiguous tickets	Urgency accuracy equalled always answering "medium", and high urgency does not mean a ticket is complex.
Escalation summary built deterministically from the ticket, history and sources	Ask the model to write it (prompt PR-04)	A model call could invent troubleshooting that did not happen. Missing context is flagged as missing instead.
Calibration split by scenario (paraphrase cluster)	Split by ticket text	My first split leaked: 87 of 90 held-out tickets had a paraphrase in the fitting set.
Groq free tier	OpenRouter free tier	The first provider throttled hard; Groq allowed the run to finish, with pacing, retries and a resumable harness.
SQLite decision log	A hosted database	Zero cost, no extra service to run from a clean checkout, and enough for an audit trail at this scale.

5.3 What the architecture does not do

It does not have a live intake: tickets enter through the evaluation runner, the scripts or the tests, and nothing accepts them from a real support inbox. It does not measure the time an agent spends. It does not rank or detect duplicate tickets, although I found that repeated texts are common (Section 9). And its single model provider is a point of failure whose effects I tested rather than eliminated (Section 8).

6. Implementation

6.1 How it was built

I built the system in the order the sprint plan set out. The first week produced the discovery workbook, the requirements document, the prompt library and the sprint plan, and the repository was created on 26 October 2026. In the second week I built normalisation, the evaluation harness, the classifier and router, retrieval, grounded drafting with guardrails, the escalation summary, the agent review page, outcome capture and the dashboard. The third week, which is still in progress as I write, covers the governance work, this report and the video. The code is organised in one module per concern (*tickets*, *classification*, *routing*, *retrieve*, *answering*, *guardrails*, *escalation*, *pipeline*, *evaluation*, the stores, and the application) so that each step can be tested without the others. The suite is run with one command, *python -m pytest tests -v*. On 11 November the 62 tests that run under Python 3.9 passed; the review API and dashboard tests need Python 3.11 and are run separately in that environment.

The evaluation harness takes an input path and an output path as arguments, not a hard-coded file name, because it will be run against a data set I have not seen. It records an error against any ticket that fails and carries on, writes one result row per ticket, and produces a metrics report. It can resume an interrupted run, reusing the tickets already processed without error, which turned out to be essential.

6.2 What was difficult

The provider. The first full run on 1 November completed 6 of 80 tickets before the free provider answered with HTTP 429 (too many requests). After I switched to Groq on 2 November, the first attempt reached 18 tickets, and I added pacing between requests, retries that honour the provider's stated reset time, and the resumable run. The final validation run took two sessions across two days because of the daily token allowance. This means that criterion A9, a single unattended run over the whole set, is not strictly met on the free tier. I report that in Section 7 and do not describe the run as a single unattended pass.

Citations. My first drafting prompt allowed the structured citation list and the inline identifiers in the text to disagree, which would have discarded sound drafts. The second version of the drafting prompt (PR-02 v2, 3 November) requires them to match, and the stored citations are now taken only from the inline identifiers that were checked against the passages retrieved for that ticket. A small probe on development tickets showed the failure mode, which is when the model cites a real document that does not support its sentence; the automatic check cannot catch that, which is why I treat it as a retrieval check only.

Evaluation hygiene. The hardest part was not a bug but a method problem. My first calibration split was by text hash, and I found that nearly every held-out ticket had a paraphrase in the fitting set. I replaced it with a split by scenario, where near-duplicates are grouped by shared content words (about 79 groups in the development set). I then discovered that intervals based on single tickets were too narrow for the same reason, so I computed every interval by resampling whole scenarios. Separately, 67 of 97 groups of identical ticket text carry different urgency labels; I report the conflict and did not relabel, because the hidden labels may conflict in the same way.

Smaller problems. The vector store printed telemetry errors and reloaded the embedding model for every ticket, which I fixed by pinning a dependency and loading once. A malformed ticket in an input file stopped the whole run; I found this while testing failure handling and fixed it (Section 8). And the pipeline reported its latency as empty, so the first report had no latency figures; I fixed the report to use the harness measurement and added a test.

6.3 What I would restructure

Four changes would make the next version better. First, a live intake: today the "degrade to review when classification fails" behaviour exists but nothing calls it outside the tests and the demonstration. Second, timing: the pipeline should record where each ticket's seconds went (waiting, retrieval, each model call), because I can infer from the two clusters in Figure 4 that the slow tickets were waiting for the provider but I did not log the cause. Third, the model prompts should live in one place with their versions and not partly inside the code, which would have made the prompt register in Appendix A simpler to keep honest. And fourth, the drafter should be told to treat a subject line that conflicts with the ticket body as unconfirmed, and the guardrails should compare an article's plan scope with the customer's plan; both are specified in PRD 2.0 but not built, because I found them after the validation set was spent and a change would have broken the comparison.

7. Evaluation

7.1 Method and data

I used three sets of tickets for different purposes. The 500 development tickets were used for analysis, for fitting calibration and for choosing between prompts and rules. The 80 validation tickets were kept for the final comparison, and I used them for no tuning. The hidden set of 120 tickets that the course will use has not been given to me, and I have run the system on it zero times; the harness takes any input path and writes the same report, so the course can run it. The validation run reported in this section used the final configuration: prompt PR-03 version 4 for classification, router version 4, and drafting prompt PR-02 version 2, with confidence calibration switched off. It processed all 80 tickets with no errors. Because of the provider's daily allowance it ran in two resumed sessions, finishing by 10 November 2026, and the report file was regenerated on 11 November to correct a latency block that had been empty.

Paraphrases of one scenario are nearly identical, so single tickets are not independent. Wherever I give an interval, it is a 95 percent interval from resampling whole scenarios (50 groups in the validation set, 76 in the development predictions), 2,000 times. A naive per-ticket interval would have been about half as wide. Where I compare two systems on the same tickets I use the paired difference.

7.2 Business outcomes

I cannot report the business outcomes the client cares about. First contact resolution, time to first reply and customer satisfaction need real tickets answered with and without the assistant, and the prototype answers none. The evaluation report therefore leaves those fields empty by design. For context, the validation tickets' own history shows what human handling achieved: first contact resolution of 46.3 percent, an escalation rate of 53.8 percent, a repeat contact rate of 21.3 percent, a mean resolution time of 441 minutes and a satisfaction average of 2.6 out of 5. These are the baseline a pilot would have to beat, not results of the system. The only business-relevant evidence I have is indirect: the system retrieves an expected article for nearly every answerable ticket, and prepares a summary for every escalation, which are the two things the interviews said would save agents' time. I did not measure the time saved.

7.3 Technical results

Table 4. Results on the 80 validation tickets (final configuration, one run)

Measure	Result	Target	How measured
Intent accuracy	91.3% (73 of 80); 95% interval 83.3 to 97.5	none stated (precision 85% per class)	Exact match with the supplied label, 22 classes
Intent precision and recall, macro average	0.94 and 0.92	85% precision per class	Per class; 4 classes fall below 85% precision, each on 3 or 4 tickets (Appendix B)
Tickets that must escalate, caught	14 of 14	all	Predicted intent in the four must-escalate intents
Urgency, exact	43.8% (33.3 to 54.9)	advisory; none	Always answering "medium" scores 43.8% (35 of 80)
Urgency, within one level	85.0%	advisory	With three levels, uninformative on its own

Retrieval hit, answerable tickets	98.1% (52 of 53)	none stated	An expected article is among the five retrieved
Citation match (proxy)	78.7%	95% (accuracy)	Cited document is an expected document. A proxy only
Latency, median and 95th percentile	6.3 s and 55.8 s	95th percentile under 3 s	End to end per ticket, under free-tier pacing
Hallucination rate; citation accuracy	Not measured	5% or lower; 95% or higher	Needs human review, not done
Guardrail blocks in this run	0	n/a	See Section 8 for the engineered demonstration
Decision log reconciliation	80 of 80	exact	80 classification and 80 routing rows; 63 draft and 17 escalation rows

The classifier is strong where the classes are distinct and weak where two labels overlap. On the validation set the weak points are API key issues (recall 2 of 6, with most errors read as security incidents, which escalate and are therefore a safe failure) and the small groups account access, authentication failure and quota or overage. With 3 to 6 tickets per weak class, I would not trust any per-class figure to within less than about 30 points, so the table of per-class values in Appendix B should be read as indications.

7.4 Changing the prompt: a result that was not clean

The first classification prompt (PR-03 v2) reached 85.6 percent intent accuracy on the development predictions (interval 77.2 to 92.8). The error analysis showed that errors were concentrated: 37 of 49 errors fell in 8 of 76 scenarios, among them the compliance request "an auditor asked for access records" misread as a data export, which would have been answered instead of escalated. I wrote a fourth version of the prompt (PR-03 v4) with explicit intent definitions and tie-break rules, and, before running it, a pre-registered hypothesis, H1: that intent accuracy would improve with no class losing more than 10 points and macro recall rising. On the development tickets the result was 93.2 percent, a paired gain of 7.6 points (interval 1.6 to 14.7), with 30 tickets fixed and 4 broken. Must-escalate tickets caught rose from 56 of 64 to 64 of 64. But **H1 was not met**: recall for authentication failure fell by 21 points. I recorded that verdict as written and adopted the prompt anyway, because the lost class is a pair of labels that overlap and both go to a person, and the gains were larger. That was a decision, not a pass, and I list it as such in the revision log.

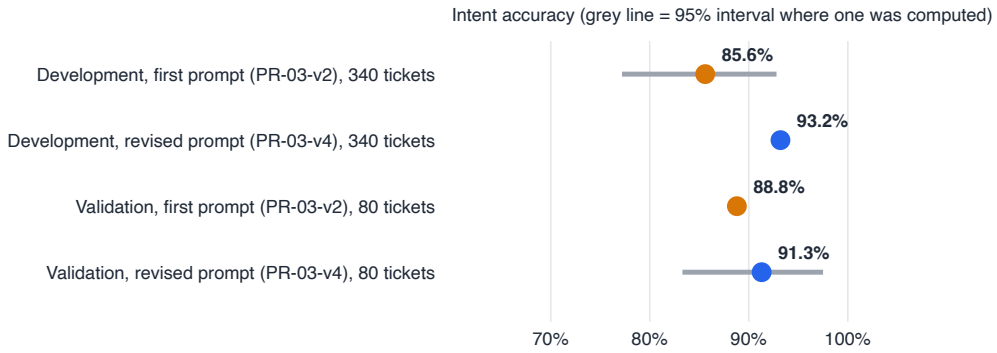


Figure 3. Intent accuracy for the two prompts. The validation difference between the prompts is two tickets (71 against 73 of 80) and is within noise; the development gain is the evidence for the change.

On validation the two prompts differ by two tickets, which I regard as no evidence either way. I used the validation set once for the final prompt and have not used it to tune anything. Because of that, I cannot test any further prompt change without a fresh, unseen set, which is a limitation of having only 80 validation tickets.

7.5 Confidence, calibration and routing

The model reports its own confidence and I hoped to turn it into a gate: answer only when calibrated confidence is above 0.80. It cannot be done with this model. PR-03 v4 emits only three distinct intent confidences (0.85, 0.95 and 0.98). On the development predictions, intent was right for 94.0 percent of the 151 tickets at 0.98 and 92.6 percent of the 188 at 0.95, a difference of 1.4 points. On validation the gap is larger (97.6 percent of 41 against 84.2 percent of 38) but rests on few tickets, and with a threshold of 0.80, below all three values, a gate would filter nothing either way. I fitted isotonic calibrations (three versions, split by scenario); the latest is flat, and none could be shown to separate right answers from wrong ones. I left calibration switched off, so with the artifact unset every ordinary ticket goes to review. That is the safest behaviour available, and it explains the routing result: the supplied labels expect 48 of the 80 validation tickets to be answered automatically and 32 to be escalated, whereas the system escalated 17 and sent 63 to an agent. Strict routing accuracy is therefore 17.5 percent (14 of 80), because the system never takes the automatic route. Of the 32 labelled for escalation, 14 were escalated and 18 went to review, where an agent sees them; none was answered automatically. I report routing accuracy and do not defend it as a success: it shows that the prototype deliberately declines to automate.

7.6 Retrieval and the drafts

Retrieval worked well. On the development tickets an expected article was among the five retrieved for 95.8 percent of answerable tickets (the first result for 88.8 percent), and on validation for 98.1 percent. Of the 63 tickets that reached drafting, 50 produced a draft with citations and 13 returned a note that evidence was insufficient; nine of those 13 were refusals because the model's text had no inline citation, a safe failure, although six of the nine concerned answerable tickets that a retry might have drafted.

Because I did not run the human review, I read the 50 drafts myself against the cited articles. I did not use the labels to judge them. The result is an informal check, not a rate: 39 drafts were sound, 7 contained a small assumption or embellishment, and 4 would mislead a reviewer who only skims (for example, a draft that told a customer to revoke a key as exposed when the ticket body described a permission-scope error, because the subject line said "exposed key"). Two drafts also described features restricted to plans the customer did not have. All 50 passed the automatic citation validation, which confirms that it checks retrieval and not whether a claim fits the ticket. The patterns, a subject line taken as fact, an article applied to a case or a plan it does not fit, and an unparsed multi-document citation, are the basis for two requirement changes in Section 9.

7.7 Latency

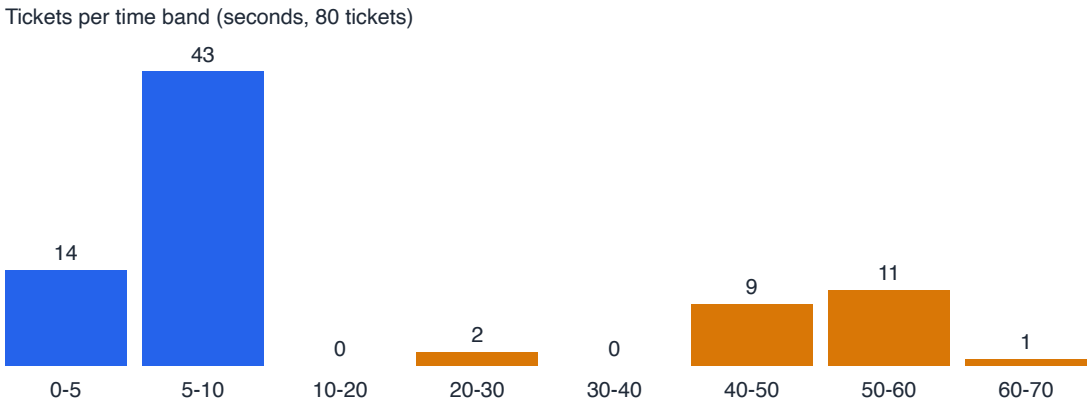


Figure 4. Per-ticket end-to-end time on the validation run. The 21 slow tickets (26 percent) occur in every kind of outcome, including escalations that make a single model call.

The median was 6.3 seconds and the 95th percentile 55.8 seconds, so the target of 3 seconds is not met. Figure 4 shows two clusters: 57 tickets under 10 seconds and 21 tickets between 42 and 62 seconds. Because the slow tickets appear in every group, including the escalations that make a single call, I infer that they waited for the provider's rate limit, but I did not log the cause, so this is an inference. The run also deliberately waited about 2.5 seconds before each model call. These are therefore end-to-end times under free-tier pacing, not a production figure, and a fair measurement needs an unpaced run on a key with higher limits, which I did not make.

7.8 Acceptance criteria

Table 5. Status of the twelve acceptance criteria

#	Criterion	Status and evidence
A1	Runs from a clean checkout	Not yet rehearsed from an empty directory on another machine; the README documents the steps.
A2	Four channels normalised	Met: validation includes all four channels, handled by one normaliser, with tests.
A3	Intent, urgency and confidence on every ticket	Met: 80 of 80 with numeric confidences.
A4	Retrieval returns identifiable passages	Met: sources are document identifiers resolving to the supplied articles.
A5	Deterministic routing threshold	The router is a pure function of the classification, and the model is called at temperature 0. I did not re-run live tickets twice to test it.
A6	Citations resolve to retrieved passages	Met at retrieval level for all 50 drafts. Whether each passage supports its sentence is not measured automatically.
A7	A guardrail blocks	Met by the input check: four engineered tickets were blocked live, with a logged block. The output guardrail never fired on the model's real output, which refused (Section 8).
A8	Decisions logged and reconciled	Met: 80 tickets, 80 classification rows and 80 routing rows.
A9	Single unattended run	Not strictly met: two resumed sessions on the free tier.
A10	Metrics report produced	Met, though first contact resolution, satisfaction and hallucination fields are empty by design.

A11	Failure handled	Met for provider disconnection, rejected key, timeout, rate limit, no retrieval and malformed input; demonstrated (Section 8).
A12	Tests pass with one command	62 tests pass under Python 3.9; two files need Python 3.11.

7.9 What I did not measure

I did not measure the hallucination rate or citation accuracy, because that needs two people to judge the drafts and I did not do it; my own read is the only substitute and is not a rate. I did not measure first contact resolution, reply time or satisfaction, because there is no pilot. I did not run an unpaced latency test, a load test, or a cost per ticket. I did not run the hidden set. And groups of 8 to 61 tickets cannot detect a gap smaller than about 20 points, so "no evidence of a gap" in the next section is weaker than "no gap".

8. Governance and risk

The Governance Framework's test is simple: pick a plausible way the system could harm a customer and ask what in the design stops it. Where my answer is a property of the model, I say so and do not count it as a control. This section covers decision logging, the risk register, the fairness audit, the guardrails, and the incident procedure. The full governance document is in the repository (*docs/governance.md*).

8.1 Decision logging

Every ticket writes a classification row and a routing row, plus either an answer-drafting row or an escalation-summary row; agent actions write a review row. All fields in the framework's minimum record exist: the identifier, time, stage, model and version, prediction and confidence, sources used, threshold applied, action, reason, guardrail results, prompt version and requirement identifiers. On the final validation run the log reconciles exactly: 80 tickets, 80 classification rows, 80 routing rows, 63 drafting rows and 17 escalation rows, each ticket with a draft or a summary and never both. Three weaknesses remain. The list of alternatives is never filled, because the classifier returns one label. The drafting row stores the full ticket text and the retrieved passages so that the review page can show them, so access to the database needs a retention rule that CloudServe has not yet agreed. And until the coverage check I did here, a classifier failure produced no row at all; it now writes an error row, and an option routes such a ticket to human review.

8.2 Risk register

Table 6. Risk register (likelihood and impact are my judgement, not measured)

ID	Risk	Likelihood / impact	Mitigation in the design, and what is not covered	Owner
R-01	Confident, incorrect answer	Medium / High	No send path; every draft reviewed; answers cite retrieved sources; four intents always escalate. Not covered: the automatic checks missed all four misleading drafts I found, and the hallucination rate is unmeasured.	Sushranth Hebbar
R-02	Private data in an outbound response	Low in the prototype / High	Tickets and drafts are blocked if they contain key-, token- or password-like strings. Not covered: names, emails, addresses, account numbers and other customers' data.	Sushranth Hebbar
R-03	Customer input treated as an instruction	Medium / Medium	The prompt marks ticket text as untrusted; the ticket and the draft are scanned for instruction-like text. Not covered: paraphrased attacks, and no injection test set was run.	Sushranth Hebbar
R-04	Some groups receive worse answers	Medium / Medium	Group audit below. Not covered: draft correctness and tone by group.	Sushranth Hebbar
R-05	Documentation goes stale	High over time / Medium	Each citation names the article. Not covered: no freshness check, and the plan-scope check is specified, not built.	Technical writer (proposed)
R-06	Model provider unavailable	High on free tier / Low in the prototype	Retries, safe results, a logged error and a resumable run; a degraded intake option routes to review. Not covered: no second provider.	Sushranth Hebbar
R-07	Latency degrades under load	High / Medium	Measured (Section 7.7): target not met. No load test.	Sushranth Hebbar

R-08	Costs rise with volume	Unknown / Medium	A drafted ticket makes two model calls and an escalation one. Not covered: no cost measured or capped.	Support Operations (proposed)

8.3 Fairness audit

I compared outcomes across the segments the framework lists, on the 80 validation tickets, using intent accuracy as the quality score. Resolution rate is unavailable because the prototype resolves nothing. Intervals in Table 7 treat tickets as independent and are therefore narrower than the scenario-based intervals used elsewhere.

Table 7. Fairness audit on validation (80 tickets; Wilson 95% intervals; variation in points from the best segment)

Segment	Tickets	Intent accuracy	Variation	Retrieval hit	Draft ready
Enterprise	8	88% (53 to 98)	-7	100% (6/6)	88%
Business	30	90% (74 to 97)	-5	100% (22/22)	73%
Standard	42	93% (81 to 98)	-2	96% (24/25)	50%
Fluent English	61	90% (80 to 95)	-5	98% (43/44)	62%
Non-fluent English	19	95% (75 to 99)	best	100% (9/9)	63%
Short (up to 24 words)	43	93% (81 to 98)	-2	96% (27/28)	65%
Long (over 24 words)	37	89% (75 to 96)	-6	100% (25/25)	59%

No intent or retrieval gap is distinguishable from noise. The variation the framework warned me to look for, worse retrieval for non-fluent English, did not appear: those tickets scored best on intent and retrieved the expected article in all 9 answerable cases. With 19 tickets I cannot exclude a gap of about 20 points, so I describe this as no evidence of a gap, which is weaker than no gap. The draft-ready rate varies by tier (88, 73 and 50 percent) but I attribute this to the mix of tickets: 11 of the 14 must-escalate tickets in the set are standard-tier, and those never receive a draft. The enterprise row, with only 8 tickets, is inconclusive. The earlier comparison by channel and region on development data showed several apparent gaps (chat and Europe intent accuracy about 10 points lower), but they traced to a few scenarios and did not reproduce on validation, where chat is level with the rest and Europe is 7 points higher. What I did not test is whether drafts differ in correctness or tone by group.

8.4 Guardrails and the demonstration

The framework lists five guardrails. The confidence floor is met: a missing or out-of-range score routes to review. Private data and instruction integrity are partial: they scan the ticket and the draft for credential-like strings and for phrases such as "ignore the previous instructions", and nothing else. Grounding is partial: it requires an inline citation to a retrieved document and does not check the claim. Tone and scope, which should stop the system promising a refund or a deadline, is missing. When a guardrail fires, the draft is blocked, the ticket goes to review, and the block is logged.

The validation run produced no block, so I tried to provoke one. Seven tickets designed to make the model repeat a credential or write an injected sentence all failed: the model refused or flagged the credential. That is good behaviour, but it is a property of the model and not a control I built, so it does not satisfy criterion A7. I therefore added a deterministic check on the ticket itself. When the subject or body contains a credential-like or instruction-like string, the pipeline skips drafting, writes a block decision with the matched category (and not the secret, which is also withheld from the log), and routes the ticket to review. Four engineered tickets submitted live to the real pipeline and model were

all blocked, with a logged block and no draft. The same patterns match none of the 580 development and validation tickets, so the check would not have changed any earlier result, but the validation run predates it.

8.5 Failure handling

I induced each failure condition. With the provider address set to a closed port, a two-ticket evaluation run finished, recorded both errors and logged them; a disconnection at the drafting stage returned a note for a human with the instruction to retry later. A wrong key, a timeout (simulated), and rate limiting (simulated, with retries) produced the same safe result. An empty retrieval and a retrieval crash produced a note for review. One malformed ticket in an input file of three stopped the whole run, which was a defect; the harness now skips and lists invalid records and processes the rest. When classification is unavailable, an option routes the ticket to review with no intent and writes a routing row, so the log still reconciles; nothing in the prototype calls it, because there is no live intake.

8.6 Incident procedure and kill switch

The incident procedure has six steps. **Detect:** an agent reports a wrong or unsafe draft, the dashboard shows a rise in blocked or insufficient-evidence outcomes, or the decision log fails to reconcile. **Contain:** unset the calibration setting if it is set, so every ticket goes to review, and stop the service; do not delete any rows. **Assess:** query the decision log by ticket for the classification, drafting and routing rows with the prompt version, the sources and the guardrail results, and decide whether the cause was the model, retrieval, a stale article, a guardrail gap or tampering, then count similar tickets. **Notify:** tell the head of support and the affected agents the same day; if a customer was told something wrong, the agent corrects it through the normal channel. **Remediate:** fix the cause, add a test that reproduces the ticket, run the suite, then restore. **Review:** record what happened, update the register, and check whether the requirements need revising.

The kill switch is the weakest part. There is no dedicated switch. The system has no path that sends to a customer, and automatic routing is off whenever the calibration setting is unset, but turning anything off means editing the environment file and restarting the service, which takes about a minute, takes effect on the next start and not on the next ticket, and is open to whoever has access to that file. I have not tested restart-to-stop as a switch. A flag read on every ticket, with a named operator, is the first thing I would add before any real deployment.

8.7 Declaration

This system must never send a message to a customer without an agent approving it, nor answer a security incident, compliance request, feature request or unclear request without a specialist. The code enforces the first by having no send path, with approval a separate agent action that re-runs the citation and safety checks, and the second by a routing rule that escalates those four intents whatever the confidence (14 of 14 caught on validation, 64 of 64 on development). The most likely way it could still cause harm is a fluent, cited draft that gets one claim wrong or applies an article to a plan it does not cover, which a busy agent approves; in my read of 50 drafts four would mislead a reviewer who skims and the automatic checks caught none. I would not deploy it without a human review of at least 50 drafts by two assessors with their agreement reported, a supervised pilot with real agents, the plan-scope and commitments checks, a runtime kill switch that has been tested, and agreed rules for data access and retention.

9. The requirements revision

I revised the product requirements document on 11 November 2026, producing version 2.0 and a revision log that records each change, what prompted it and who agreed it. I was the only author and reviewer, so the "agreed by" column names me; no outside approver was available, and the document says it is not yet approved. The revision changed eight requirements, added one, removed none, and rewrote the success-measures section. The changes, and the evidence that caused each, are in Table 8.

Table 8. What changed in the requirements and what prompted it

Requirement	Change	What prompted it
FR-02 documentation	Tell the agent when an article's "applies to" scope does not match the customer's plan (specified, not built)	Two of the 50 drafts described features restricted to Business and Enterprise plans to Standard-plan customers.
FR-03 grounded draft	Answer what the ticket body says; treat a conflicting subject as unconfirmed (specified, not built)	Six of 50 drafts took a mismatched subject as fact; one invented a slow database restore.
FR-04 agent review	The system has no send path; "auto-respond" means only that a draft is ready	The route name was read as "sent automatically" and confused the interpretation of routing accuracy.
FR-05 insufficient evidence	Four intents always escalate	With the first prompt 7 of 17 compliance requests were misread as data exports and would have been answered.
FR-09 classification	Intent confidence gates drafting only if calibrated; urgency is advisory with no accuracy target	Urgency accuracy equalled always answering "medium"; confidence did not separate right from wrong.
NFR-01 latency	Target set at 95th percentile under 3 seconds; first measurement recorded as not met	The "to be decided" had to be resolved to evaluate against it.
NFR-03 accuracy	Hallucination 5% or lower and citation accuracy 95% or higher by human review; automatic checks are retrieval checks only	All 50 drafts passed the automatic checks, including the four I judged misleading.
NFR-06 fairness	Gap under 5 points, intervals from scenario resampling, small groups inconclusive	Per-ticket intervals were too narrow because paraphrases are near-duplicates.
NFR-08 (new)	Labels never rewritten; calibration split by scenario; decisions on development data only	My first split leaked: 87 of 90 held-out tickets had a paraphrase in the fitting set.
Section 8 measures	Split into offline measures and pilot-only measures	The prototype never sends, so first contact resolution, reply time and satisfaction cannot be measured offline.

9.1 Which assumptions failed

Of the three assumptions I wrote down, the first held largely: the documentation is usable, with an expected article retrieved for 52 of 53 answerable validation tickets, but the weakness was applicability, not coverage. The second held only in part: ticket bodies are usually enough, but subjects often disagree with them and the drafter followed the subject in six of 50 drafts. The third, that agents will review drafts properly, is untested because there was no pilot. The three assumptions I had not written down all failed. A numeric confidence could not gate automatic answers: on the development tickets the mean confidence was 0.958 when the intent was right and 0.934 when

wrong. Urgency could not be predicted from the text: exact accuracy was 47.5 percent on development and 43.8 percent on validation. And a free provider could not run the whole evaluation in one unattended pass.

9.2 What the revision taught me

I assumed that confidence and urgency would work as control signals, with a score saying which tickets could be answered and urgency saying which needed a person. The data showed the opposite. What the interviews asked for was help finding the right article and a draft an agent can check, so the real problem was making the agent's review reliable, not deciding which tickets to automate. I also learned how much of an evaluation depends on how the data is split. Counting the repeated-text groups in the first week, where 67 of 97 groups had conflicting urgency labels, and checking the always-medium baseline, would have shown that an urgency target was not meaningful; and a small calibration check on 50 tickets would have shown the same about confidence. I did not do either until the second week.

I decided not to make some changes. I did not relabel the conflicting urgency groups, because the hidden labels may conflict in the same way and a system tuned to cleaned labels would not match them. I did not activate calibration, because the gate filtered nothing. I did not rewrite the knowledge base, because retrieval already finds an article for 98 percent of answerable tickets and the issue is a scope check. I kept the revised classification prompt although its pre-registered hypothesis failed, and I recorded the verdict as written. Each of these is in the "not changed" table of the revision log with the reason and the condition that would reopen it.

10. Conclusions

I built an assistant that helps CloudServe's agents, not a chatbot that replaces them, and the evidence I gathered supports that choice more than it supports the original request. On the validation set the assistant classified intent correctly for 91.3 percent of tickets, retrieved an expected article for 98.1 percent of answerable ones, escalated every ticket that policy says must go to a specialist, kept a decision log that reconciled exactly, and degraded safely in every failure condition I could induce. It never sends anything to a customer, and I think that is the right design for a team whose head of support named a confident wrong answer as a reason the project could fail.

I am less certain about what it achieves. I have no evidence that it improves first contact resolution, reply time or satisfaction, because nothing was tested with agents on real tickets. I have no measured hallucination rate or citation accuracy, so I cannot say it meets the 5 percent and 95 percent targets, and the one informal read I made suggests that roughly one draft in twelve would mislead a reviewer who does not check carefully, and that the automatic checks cannot see which. Its latency is far above the 3-second target under free-tier pacing, and I do not know what it would be on a paid key. The hidden evaluation set has not been run. These gaps are the content of the next step.

10.1 What I would do next

1. **Measure the drafts.** Run the two-assessor review of the 50 drafts with agreement reported. The packet, blank score sheets and scoring script are in the repository. This would turn two empty targets into measurements.
2. **Fix the two draft problems I found.** Treat a subject that conflicts with the body as unconfirmed, and compare an article's plan scope with the customer's plan. Both are specified in the requirements, and the fix needs a new, unseen evaluation set, since the validation set has been used.
3. **Close the guardrail gaps.** Add the commitments check (no promises about refunds or timelines), widen the private-data patterns beyond credentials, and test paraphrased injection attempts.
4. **Build a live intake and a runtime kill switch.** A flag read on every ticket, a named operator, and a test that it works.
5. **Run a supervised pilot.** With a handful of Tier One agents and real tickets, measuring review time, edits, rejections, first contact resolution and satisfaction, and comparing by tier and fluency, because enterprise customers already have the lowest first contact resolution in the data.
6. **Measure latency fairly.** An unpaced run on a key with higher limits, with the causes of delay logged per ticket.

10.2 What remains uncertain

Whether the prompt gain holds beyond the roughly nine scenarios that produced it. Whether agents catch the misleading drafts. Whether the fairness result for non-fluent customers survives a larger sample; with 19 tickets it could hide a gap of about 20 points. Whether the urgency labels reflect customer circumstances that are not in the ticket text, which would explain why identical texts carry different labels. And whether the free-tier model I used is representative of the model CloudServe would run. I would want to resolve these with a pilot and an unpaced evaluation before recommending that the assistant touch a real queue.

Declaration of AI tool use

I used Claude Code, Anthropic's command-line coding assistant running Claude models, throughout this project, and the use was substantial. The assistant did the following under my direction: it wrote and edited code in the repository, including the pipeline, guardrails, evaluation harness, review application and tests; wrote the analysis scripts for calibration, error analysis, confidence intervals and the fairness audit; ran experiments and the evaluation runs; read the 50 validation drafts against the cited articles; drafted the analysis documents in the repository (the development error analysis, the governance documents, the credential scan and this report); and drafted text in the stage workbooks and the effort log, including the estimated hours. Beyond the assistant, I relied on the supplied course pack (the transcripts, data sets, specifications and templates), on open-source libraries (Chroma, a LangChain text splitter, the MiniLM sentence embedding model, FastAPI, requests, python-dotenv and python-docx), and on the Qwen 3.8 27-billion-parameter model served by Groq as the language model inside the system. The prompts that run in the system were drafted with the assistant and are listed in Appendix A.

The course asks that the problem statement, the evaluation interpretation and the reflection come from my own judgement. The problem statement in Section 3.7 is the text from my discovery workbook. The interpretation and reflection in Sections 7, 9 and 10 were drafted by the assistant from the workbooks and the result files in the repository. Every number in this report comes from a file in the repository, and each is stated with the data it was measured on.

Examples where I overrode or corrected the assistant, or where its output needed correcting, are these. When it told me that a validation run did not exist, I had in fact started it, and I corrected the record. When the course documents gave the wrong deadline, I corrected it to 22 November. I decided not to carry out the two-assessor review of the drafts for lack of time, even though the assistant had built the review packet, so the hallucination rate and citation accuracy are reported as not measured and the assistant's own informal read is labelled as such. I chose which of its suggested guardrail changes to build (the input check and the logging of classifier failures) and which to leave as known gaps. The assistant's first guardrail demonstration used an invented key that resembled a real one and was rejected by the repository host's secret scanner, so I approved rewriting my unpushed commits to remove it; a scan of the whole history found no real credential. The hours in the effort log are estimates that the assistant produced from the commit history and the plan, and are not a timesheet.

Author to confirm before submission, then delete this box. This declaration states only what the working record shows. The course asks you to say where you overrode or corrected the tool and where the judgement was yours. Add or change anything that does not match what you did yourself: which code and which workbook text you wrote or rewrote, whether you wrote the Stage 1 discovery workbook, which parts of this report you edited, and whether you checked the effort-log hours against your own memory.

Appendix A. Prompt register

ID	Purpose	Version and history	Used at runtime
PR-01	Design-time prompt used while planning	V1.0, no later revisions	No
PR-02	Grounded reply drafter: answer only from the retrieved passages, cite each claim inline, say when evidence is insufficient	V1.0 initial. V2.0 (3 November 2026): inline citations and the structured list must match; stored citations come only from inline identifiers checked against the retrieved sources	Yes (v2.0)
PR-03	Classifier: intent from 22 labels, urgency, confidences, urgency evidence excerpt, ambiguity flag	V1.0 initial. V2 (1 November): evidence excerpt or ambiguity flag required. V3 (4 November): active-problem wording; tested on 16 development tickets, urgency matched 10 of 15 for both v2 and v3, not adopted. V4 (6 November): intent definitions and tie-break rules; development intent accuracy 85.6% to 93.2%; runtime default since 9 November (H1 not fully met)	Yes (v4)
PR-04	Escalation summary prompt	V1.0. The implemented summary is assembled deterministically from the ticket, history and retrieved sources, with no model call	No

Appendix B. Full result tables

Table B1. Intent classification per class, validation set (80 tickets, final run)

Intent	Tickets	Precision	Recall
account access	4	75%	75%
api key issue	6	100%	33%
api usage question	4	100%	100%
authentication failure	3	67%	67%
billing query	10	91%	100%
compliance request	1	100%	100%
configuration help	3	75%	100%
data export	1	100%	100%
data residency	6	100%	100%
database issue	1	100%	100%
deployment failure	5	100%	100%
feature request	3	100%	100%
integration help	3	100%	100%
onboarding	4	100%	100%
performance degradation	3	100%	100%
quota or overage	2	100%	50%
rate limit	1	100%	100%
rollback request	6	100%	100%
security incident	4	57%	100%

sso configuration	1	100%	100%
unclear request	6	100%	100%
webhook issue	3	100%	100%

Macro-averaged precision is 0.94 and recall 0.92. Classes with fewer than about 6 tickets give wide uncertainty; the four classes below 85 percent precision (account access, authentication failure, configuration help, security incident) each have 3 or 4 tickets. Security incident precision is low because tickets that other labels describe were predicted as security incidents, which escalate.

Table B2. Routing outcomes on validation: label-expected route against route taken

	Escalated by system	Sent to review	Auto-responded
Labelled escalate (32)	14	18	0
Labelled auto-respond (48)	3	45	0

Table B3. Drafting outcomes on validation

Outcome	Tickets
Escalated, summary prepared, no draft	17
Draft ready for agent review	50
Note for a human (insufficient evidence); 9 of 13 were refusals for missing inline citation	13
Blocked by a guardrail	0

Table B4. Latency by outcome, validation run

Group	Tickets	Median	95th percentile	Over 30 s
Escalated (classification only)	17	2.9 s	54.9 s	4
Draft ready	50	6.3 s	53.9 s	14
Needs human	13	6.1 s	57.1 s	3
All	80	6.3 s	55.8 s	21

Table B5. Informal read of the 50 validation drafts (not a measured rate)

Verdict	Drafts
Sound: every claim traces to the cited text and fits the ticket	39
Minor: supported text with a small assumption or embellishment	7
Would mislead a reviewer who only skims	4

Appendix C. Requirement traceability

Requirement	Code	Test file
FR-01 ticket and context	src/tickets.py, src/review_store.py	test_tickets, test_agent_review_api
FR-02 documentation	src/retrieve.py	test_retrieve
FR-03 grounded draft	src/answering.py, src/guardrails.py	test_answering
FR-04 agent review, no send path	src/app.py, src/review_store.py, static/review.html	test_agent_review_api
FR-05 insufficient evidence, must-escalate intents	src/routing.py, src/guardrails.py	test_classification_routing, test_answering
FR-06 escalation summary	src/escalation.py	test_escalation
FR-07 sources and decisions recorded	src/logging_store.py, src/pipeline.py, src/review_store.py	test_logging_store, test_answering
FR-08 outcomes and feedback	src/outcome_store.py, src/dashboard.py	test_outcomes, test_dashboard
FR-09 intent, urgency, confidence	src/classification.py, src/routing.py, src/calibration.py	test_classification_routing, test_calibration, test_review_urgency
NFR-01 latency	src/evaluation.py (measurement)	test_evaluation
NFR-04, NFR-05 privacy, auditability	src/guardrails.py, src/logging_store.py	test_answering, test_logging_store
NFR-06 fairness	scripts/analyze_accuracy_uncertainty.py, scripts/governance_fairness_audit.py	run on result files
NFR-08 evaluation integrity	src/scenarios.py, src/evaluation.py	test_calibration, test_evaluation

Appendix D. Workbooks, logs and how to reproduce

Submitted with this report. The five stage workbooks (discovery, requirements version 2.0, prompt library, sprint plan, and the requirements revision log) and the effort log, in the workbooks folder; the complete repository in the source code folder. Supporting documents in the repository: *docs/governance.md*, *docs/governance_coverage_check.md*, *docs/validation_draft_review.md*, *docs/validation_latency_and_fairness.md*, *docs/development_error_analysis.md*, *docs/accuracy_uncertainty_and_fairness.md*, *docs/urgency_and_label_noise.md*, *docs/credential_scan.md*.

Effort. The effort log totals about 80 hours across three weeks to 11 November, with the report, video and packaging still to complete. Task entries between 26 October and 11 November were reconstructed on 11 November from the commit history and project notes, and the hours are estimates, not a timesheet. Build and revision took far longer than planned (about 64 hours against 25 planned for the original backlog items), because calibration, error analysis, the routing policy, the review workflow and repeated runs under the free-tier quota were not in the plan.

Dates of evaluation runs. First validation run 1 November 2026 (6 of 80 tickets before rate limiting); paced run 2 November (80 of 80, prompt PR-03 v2, intent 88.8%); final run in the final configuration completed by 10 November (80 of 80, no errors, intent 91.3%), report regenerated 11 November. The hidden 120-ticket evaluation set: not run by me, zero times.

Reproduction. Install the requirements, copy *.env.example* to *.env* and add a model key, run *python -m pytest tests -v* for the tests, *python scripts/run_evaluation.py --input <tickets.json> --output <results.jsonl> --processor src.pipeline:process_ticket* for an evaluation run (add *--resume* to continue an interrupted run), and *uvicorn src.app:app* for the review page. The demonstrations are *scripts/demo_guardrail_ticket.py* and *scripts/demo_failure_modes.py*.